

COMPLETE GUIDE TO

Generative AI & Agentic AI

From LLMs and Prompt Engineering to Agent Architectures
and Multi-Agent Systems

Prepared: April 2026

Part 1

Part 2

17+

Key Concepts

Table of Contents

PART 1 — GENERATIVE AI

- 1. What is Generative AI?
- 2. How LLMs Work (Tokens, Context Window, Transformers)
- 3. Popular Gen AI Models (GPT, Claude, Gemini)
- 4. Prompt Engineering Basics
- 5. Gen AI Use Cases
- 6. API Integration with Gen AI
- 7. Limitations & Hallucinations
- 8. Ethics & Responsible AI

PART 2 — AGENTIC AI

- 9. What is Agentic AI?
- 10. Agents vs Chatbots
- 11. Agent Architecture (Planning, Reasoning, Tool Use)
- 12. Function Calling & Tool Use
- 13. Agent Frameworks (LangChain, CrewAI)
- 14. Multi-Agent Systems
- 15. Agent Memory (Short-term, Long-term)
- 16. Building Your First Agent
- 17. Real-world Use Cases
- 18. Limitations & Safety Guardrails

Generative AI

Understanding LLMs, Prompting, APIs, and Responsible Use

1 What is Generative AI?

Generative AI refers to a class of artificial intelligence systems capable of producing new content — text, images, audio, video, code, or structured data — by learning patterns from vast amounts of training data. Unlike traditional AI that classifies or predicts from fixed categories, generative models create novel outputs based on statistical patterns learned during training.

At its core, a generative AI model learns a probability distribution over data. Given a prompt or seed input, it samples from that distribution to produce plausible, coherent, and contextually relevant outputs.

Key Characteristics

- **Creative output:** Can generate human-like text, realistic images, lifelike audio
- **Pattern learning:** Learns structure from billions of data examples
- **Conditional generation:** Output is guided by a user prompt or context
- **Probabilistic nature:** Same input can yield varied outputs
- **Emergent capabilities:** Large models exhibit abilities not explicitly trained

Type	Traditional AI	Generative AI
Task	Classify, predict, detect	Create, generate, compose
Output	Label, number, category	Text, image, code, audio
Training	Labeled datasets	Large unlabeled corpora
Examples	Spam filter, fraud detect	ChatGPT, DALL-E, Sora

■ **Key Insight:** Generative AI does not "understand" in the human sense — it models the statistical distribution of human-generated content and samples from it to create new outputs.

Tokens

Tokens are the fundamental units LLMs process. A token is roughly 3–4 characters or 0.75 words in English. The process of splitting text into tokens is called **tokenization**. Models like GPT-4 use Byte Pair Encoding (BPE), which merges frequent character sequences into single tokens.

Word	"Generative" → ["Gener", "ative"] (2 tokens)
Number	"12345" → ["123", "45"] (2 tokens)
Rule	1 token ≈ 4 chars (English); non-English languages use more tokens per word
Impact	Token count determines API cost and context window usage

Context Window

The context window is the maximum number of tokens a model can process in a single call — both the input prompt and the generated output combined. Everything within this window is "visible" to the model; text outside is forgotten.

- **GPT-4o**: 128,000 tokens (~300 pages of text)
- **Claude 3.5 Sonnet**: 200,000 tokens (~450 pages)
- **Gemini 1.5 Pro**: 1,000,000+ tokens (multi-modal long context)
- Larger context = better for long documents, codebases, and conversations
- Cost scales with tokens processed

Transformer Architecture

Transformers, introduced in the 2017 paper "Attention Is All You Need", are the backbone of modern LLMs. The architecture replaces recurrence with **self-attention**, enabling parallel processing of sequences and capturing long-range dependencies.

Component	Function	Why It Matters
Tokenizer	Splits text into numeric token IDs	Converts human text to model input
Embedding Layer	Maps token IDs to dense vectors	Captures semantic meaning
Self-Attention	Each token attends to all others	Captures context & relationships
Multi-Head Attention	Multiple attention operations in parallel	Captures different relationship types
Feed-Forward Network	Non-linear transformations per token	Learns complex patterns
Layer Normalization	Stabilizes training	Enables deeper, stable networks

Output Head (LM)	Maps vectors to vocabulary logits	Predicts next token probabilities
------------------	-----------------------------------	-----------------------------------

■ **Training:** LLMs are pre-trained on next-token prediction (predict the next word given all previous words) over trillions of tokens. This "self-supervised" learning requires no human labels and scales with data and compute.

The generative AI landscape is dominated by a few large language model families. Each has distinct strengths, architectures, and design philosophies.

Model Family	Creator	Key Models	Strengths	Context
GPT Series	OpenAI	GPT-4o, o1, o3	Broad capability, code, reasoning, vision	128K
Claude	Anthropic	Claude 3.5 Sonnet, Opus 4	Safety, long context, writing, analysis	200K
Gemini	Google DeepMind	Gemini 1.5 Pro, 2.0 Flash	Multimodal, search integration, 1M context	1M+
Llama	Meta AI	Llama 3, 3.1, 3.3	Open source, self-hostable, customizable	128K
Mistral	Mistral AI	Mistral Large, Mixtral	Efficient, multilingual, European	128K
Command R	Cohere	Command R+	Enterprise RAG, grounding, low latency	128K

Model Selection Guide

- **GPT-4o / o1:** Best for complex reasoning, coding, and OpenAI ecosystem integration
- **Claude:** Best for long documents, nuanced writing, safety-critical applications
- **Gemini:** Best for multimodal tasks, Google Workspace integration, ultra-long context
- **Llama 3:** Best when you need an open-source, self-hosted, or fine-tuned model
- **Mistral:** Best for cost-efficient European-compliant deployments

4 Prompt Engineering Basics

Prompt engineering is the practice of crafting effective inputs to guide LLM outputs. Since models are sensitive to phrasing, a well-designed prompt dramatically improves output quality, consistency, and relevance.

Core Prompting Techniques

Technique	Description	Example
Zero-Shot	Ask directly without examples	"Summarize this article in 3 bullets."
Few-Shot	Provide examples in the prompt	"Q: What is ML? A: ... Q: What is DL? A: ..."
Chain-of-Thought	Ask model to reason step-by-step	"Think step by step: If A > B and B > C, then..."
Role Prompting	Assign a persona to the model	"You are an expert data scientist. Explain..."
Instruction Tuning	Give precise formatted instructions	"Return ONLY a JSON object with keys: name, age"
Retrieval-Augmented	Inject retrieved context into prompt	Paste relevant docs + "Answer based only on above:"

Prompt Structure Best Practices

- **Be specific:** Vague prompts yield vague results; specify format, length, tone
- **Set context:** Tell the model who it is, who it's talking to, and why
- **Use delimiters:** Separate instructions from content with XML tags or triple quotes
- **Specify output format:** Ask for JSON, markdown, numbered list, etc.
- **Iterate:** Prompt engineering is an empirical process; test and refine

Example — Structured Prompt:

System: You are a helpful assistant that extracts entities from text.

User: Extract all person names and their roles from the following text.

Return a JSON array like: [{"name": "...", "role": "..."}]

Text: <text>John Smith, the CEO of Acme Corp, met with Mary Roe, Head of Engineering.</text>

5 Gen AI Use Cases

Generative AI is transforming virtually every industry. Its ability to generate, summarize, classify, and reason over language makes it applicable anywhere humans communicate or process information.

Domain	Use Cases	Example Tools
Software Dev	Code generation, code review, bug fixing, test writing, documentation	GitHub Copilot, Cursor, Claude
Content & Media	Blog writing, ad copy, video scripts, SEO content, translation	Jasper, Copy.ai, ChatGPT
Customer Service	Chatbots, FAQ answering, ticket routing, sentiment analysis	Intercom, Zendesk AI, ServiceNow
Healthcare	Clinical note summarization, drug research, patient Q&A;	Nuance DAX, Med-PaLM
Legal	Contract analysis, document drafting, legal research, compliance	Harvey AI, CoCounsel
Education	Personalized tutoring, quiz generation, essay feedback	Khan Academy AI, Duolingo
Finance	Report generation, earnings summaries, fraud detection, risk analysis	BloombergGPT, Fin-BERT
Data & Analytics	Text-to-SQL, insight narration, data cleaning, anomaly detection	Databricks AI, Tableau AI

Most LLMs are accessible via REST APIs. Understanding the API structure is essential for building applications that leverage generative AI capabilities.

Common API Concepts

Endpoint	POST /v1/messages (Anthropic), POST /v1/chat/completions (OpenAI)
Authentication	API key in Authorization header: "Authorization: Bearer sk-..."
Model	Specify which model version to use (e.g., "claude-sonnet-4-6")
Messages	Array of {role, content} objects: "system", "user", "assistant"
Temperature	Controls randomness: 0.0 = deterministic, 1.0 = creative, 2.0 = chaotic
Max Tokens	Maximum tokens in the response; controls length and cost
Streaming	Server-sent events (SSE) for token-by-token streaming responses
Stop Sequences	Strings that signal the model to stop generating

Python Example — Anthropic API:

```
import anthropic
client = anthropic.Anthropic(api_key="your-api-key")

message = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": "Explain quantum computing"}]
)
print(message.content[0].text)
```

Rate Limits and Cost Management

- **Rate limits:** APIs enforce Requests Per Minute (RPM) and Tokens Per Minute (TPM)
- **Pricing:** Typically charged per 1M input tokens and per 1M output tokens separately
- **Caching:** Many APIs offer prompt caching to reduce cost for repeated prompts
- **Batching:** Batch APIs process large volumes at ~50% cost with async delivery
- **Monitoring:** Track token usage and costs via API dashboard or usage headers

7 Limitations & Hallucinations

Despite remarkable capabilities, LLMs have fundamental limitations that practitioners must understand and mitigate. The most well-known is **hallucination** — the tendency to generate plausible-sounding but factually incorrect information.

Types of Hallucinations

Type	Description	Example
Factual	States false facts confidently	Invents a citation, wrong date, false statistic
Entity Confusion	Mixes up similar people/places	Attributes a quote to wrong person
Temporal	Wrong about time-sensitive info	States outdated model as "latest"
Intrinsic	Contradicts provided source text	Summarizes document incorrectly
Extrinsic	Adds info not in source	Invents details not mentioned in input

Other Key Limitations

- **Knowledge cutoff:** Training data has a fixed cutoff date; no real-time knowledge
- **Context length:** Cannot process arbitrarily long documents in one call
- **Reasoning errors:** Struggles with complex multi-step mathematical or logical reasoning
- **Inconsistency:** Same prompt may yield different answers across runs
- **Bias:** Inherits societal biases present in training data
- **Verbosity:** Tendency to over-explain or pad responses
- **Prompt injection:** Malicious user inputs can override system instructions

■ **Mitigation Strategies:** Use RAG (retrieval-augmented generation) for factual queries, add citation requirements, implement output validation, set lower temperatures for factual tasks, and use post-generation fact-checking pipelines.

8 Ethics & Responsible AI

As generative AI becomes pervasive, ethical deployment is not optional — it is a professional and societal responsibility. Responsible AI encompasses fairness, transparency, privacy, safety, and accountability.

Principle	Concern	Best Practice
Fairness	Models may discriminate against groups	Audit outputs for bias; diverse training data
Transparency	Users may not know AI generated content	Label AI-generated content; explainable outputs
Privacy	User data used in training without consent	Data anonymization; avoid PII in prompts
Safety	Harmful, toxic, or dangerous outputs	Content filtering; red-teaming; RLHF alignment
Accountability	Who is responsible for AI errors?	Human-in-the-loop for high-stakes decisions
Copyright	Models may reproduce copyrighted material	License-aware training; output filtering
Misinformation	AI-generated fake news, deepfakes	Watermarking; detection tools; media literacy
Environmental	High energy consumption for training	Efficient models; green data centers

Frameworks and Regulations

- **EU AI Act:** Risk-based regulation classifying AI systems into prohibited, high-risk, and minimal-risk
- **NIST AI RMF:** US National Institute of Standards framework for AI risk management
- **Anthropic Constitutional AI:** Training models to follow principles and refuse harmful requests
- **OpenAI Usage Policies:** Prohibited use cases including weapons, CSAM, disinformation
- **ISO/IEC 42001:** International standard for AI management systems

Agentic AI

Agents, Tool Use, Frameworks, Multi-Agent Systems & Safety

9

What is Agentic AI?

Agentic AI refers to AI systems that can autonomously plan, reason, and take actions to accomplish goals — going beyond single-turn question answering. An **agent** is an LLM-powered system that perceives its environment, makes decisions, and executes actions (like browsing the web, writing files, or calling APIs) in a loop until a goal is achieved.

The shift from "chatbot" to "agent" is a shift from **reactive** (respond to each message) to **proactive** (pursue a goal through a sequence of actions). Agents can operate autonomously for minutes, hours, or days.

Core Agent Properties

- **Goal-directed:** Works toward a defined objective over multiple steps
- **Autonomous:** Makes decisions without step-by-step human guidance
- **Tool-using:** Can interact with external systems (web, APIs, files, databases)
- **Memory:** Maintains state and context across multiple actions
- **Adaptive:** Adjusts its plan based on feedback and intermediate results
- **Iterative:** Observe → Think → Act loop continues until goal is met

■■ **The Agent Loop:** Receive Goal → Plan Steps → Choose Tool/Action → Execute → Observe Result → Reflect → Repeat or Finish

10 Agents vs Chatbots

Chatbots and agents both use LLMs, but differ fundamentally in how they operate and what they can accomplish.

Dimension	Chatbot	Agent
Interaction	Turn-by-turn conversation	Autonomous multi-step execution
Duration	Single response per query	Minutes to hours per task
Tools	None or limited (e.g., web search)	Full tool suite (APIs, files, browsers, code)
Planning	No explicit planning	Decomposes goals into sub-tasks
Memory	Conversation history only	Persistent long-term memory stores
Error Handling	None; user must retry	Self-corrects based on tool feedback
Autonomy	Human drives every step	Human sets goal; agent executes
Example	Customer support bot	Research agent, coding agent, data analyst

11 Agent Architecture (Planning, Reasoning, Tool Use)

A modern AI agent consists of several interacting components. The LLM serves as the "brain" while specialized modules handle memory, tools, and execution.

Core Architectural Components

LLM Core	The reasoning engine. Interprets instructions, plans steps, selects tools, and generates outputs.
Profile / Persona	System prompt defining agent's role, personality, constraints, and objectives.
Memory Module	Short-term (context window) + Long-term (vector DB, files) storage.
Tool Registry	Collection of callable functions/APIs the agent can invoke.
Planner	Decomposes complex goals into ordered sub-tasks (ReAct, ToT, MCTS).
Executor	Runs tool calls, collects outputs, feeds results back to the LLM.
Evaluator	Checks if the goal has been achieved; triggers replanning if not.
Safety Layer	Filters harmful actions, enforces constraints, limits blast radius.

Reasoning Strategies

Strategy	Description	Best For
ReAct	Interleaves Reasoning + Acting in each step	General tool-using agents
Chain-of-Thought	Explicit step-by-step thinking before answering	Complex reasoning tasks
Tree of Thoughts	Explores multiple reasoning paths as a tree	Creative or optimization problems
Reflexion	Agent critiques its own outputs and self-improves	Tasks requiring high accuracy
Plan-and-Execute	Creates full plan first, then executes step by step	Long-horizon complex tasks

12 Function Calling & Tool Use

Function calling (or "tool use") is the mechanism by which an LLM can request execution of external code or APIs. The model does not execute code itself — it outputs a structured JSON request, which the application executes and returns results to the model.

How Function Calling Works

Step 1	Developer defines tools as JSON schemas: name, description, parameters
Step 2	Tool definitions are included in the API request alongside the user message
Step 3	LLM decides if a tool is needed; outputs a tool_call block instead of text
Step 4	Application parses the tool_call, executes the function, gets the result
Step 5	Result is sent back to LLM as a tool_result message
Step 6	LLM uses the result to continue or produce a final answer

Tool Definition Example (JSON Schema):

```
{
  "name": "get_weather",
  "description": "Get current weather for a city",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "City name"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"]
      }
    },
    "required": ["city"]
  }
}
```

Common Tool Categories

- **Web & Search:** Web search, URL fetching, news retrieval
- **Code Execution:** Python REPL, JavaScript sandbox, shell commands
- **File Operations:** Read/write files, list directories, parse documents
- **Database:** SQL queries, NoSQL lookups, vector similarity search
- **APIs:** REST/GraphQL calls to external services (CRM, calendar, email)
- **Browser Automation:** Playwright/Selenium for web scraping and UI testing
- **Communication:** Send emails, Slack messages, create calendar events

13 Agent Frameworks (LangChain, CrewAI)

Agent frameworks abstract the complexity of building LLM-powered agents, providing pre-built components for tool use, memory, planning, and multi-agent orchestration.

Framework	Language	Focus	Best For	Key Feature
LangChain	Python/JS	General-purpose agents & chains	RAG, tool agents, pipelines	LCEL, LangGraph integration
LangGraph	Python	Stateful graph-based agents	Complex workflows, branching	Cyclic graphs, human-in-loop
CrewAI	Python	Role-based multi-agent systems	Team simulations, research	Agent roles, task delegation
AutoGen	Python	Conversational multi-agent	Code generation, debate	Agent-to-agent conversation
Semantic Kernel	C#/Python	Enterprise AI orchestration	Microsoft ecosystem	Skills, planners, connectors
Haystack	Python	NLP pipelines & RAG	Document Q&A, search	Pipelines, retrievers
Agno	Python	Lightweight agent library	Simple tool-using agents	Minimal boilerplate

LangChain — Key Concepts

- **LCEL (LangChain Expression Language):** Declarative pipeline composition with `|` operator
- **Chains:** Linear sequences of LLM calls and transformations
- **Agents:** LLM + tool router that decides which tools to call
- **Memory:** ConversationBufferMemory, VectorStoreMemory, etc.
- **Retrievers:** Fetch relevant documents from vector stores for RAG

CrewAI — Key Concepts

- **Agents:** Role-based LLM instances (Researcher, Writer, Analyst)
- **Tasks:** Discrete units of work assigned to agents
- **Crew:** A team of agents working toward a shared goal
- **Process:** Sequential, hierarchical, or consensual execution modes
- **Tools:** Shared or agent-specific tool sets

14 Multi-Agent Systems

Multi-agent systems (MAS) consist of multiple AI agents interacting to solve problems that exceed the capability or context of a single agent. Each agent specializes in a domain; coordination produces emergent intelligence.

Topologies

Topology	Description	Use Case
Hierarchical	Orchestrator delegates to worker agents	Complex research, coding pipelines
Pipeline	Agents pass outputs sequentially	Content generation, data transformation
Parallel	Agents work concurrently on sub-tasks	Large-scale data analysis
Peer-to-peer	Agents communicate directly with each other	Debate, fact-checking, consensus
Swarm	Many simple agents with emergent behavior	Optimization, simulation
Market/Auction	Agents bid for tasks based on capability	Resource allocation, scheduling

Challenges in Multi-Agent Systems

- **Coordination:** Preventing conflicting actions and deadlocks between agents
- **Communication:** Designing efficient inter-agent message protocols
- **Trust:** Verifying agent outputs before downstream consumption
- **Cost:** Multiple LLM calls multiply token costs significantly
- **Error propagation:** Early agent errors cascade through the pipeline
- **Observability:** Tracing and debugging distributed agent interactions

■ **Real Example — Research Agent Crew:** Planner Agent breaks the research question into sub-queries → 3 Research Agents search the web in parallel → Synthesis Agent combines findings → Critic Agent challenges conclusions → Writer Agent produces the final report.

15 Agent Memory (Short-term, Long-term)

Memory is what separates stateful, intelligent agents from stateless question-answerers. Agents must remember past actions, user preferences, learned facts, and intermediate results across a task and across sessions.

Memory Type	Storage	Scope	Use Case	Technology
Sensory	Raw inputs	Immediate	Current perception	Input buffer
Short-term / Working	Context window	Current session	Conversation history, task state	LLM context
Episodic	External DB	Multi-session	Past conversations, user history	Vector DB + timestamps
Semantic	Vector store	Long-term facts	Domain knowledge, user profile	Pinecone, Chroma, Weaviate
Procedural	Code/tools	Persistent skills	How to perform tasks	Tool library, code files
Shared / Team	Shared DB	Multi-agent	Agent communication	Redis, PostgreSQL

Memory Management Strategies

- **Summarization:** Compress old conversation turns to save context space
- **Retrieval:** Embed memories as vectors; retrieve only relevant ones per query
- **Hierarchical:** Recent = full detail; older = summaries; ancient = compressed
- **Selective forgetting:** Delete irrelevant or outdated memories periodically
- **External KV store:** Redis or DynamoDB for fast key-value fact retrieval

16 Building Your First Agent

Building a simple agent requires: an LLM, a set of tools, and an execution loop. Below is the conceptual walkthrough and minimal code structure.

Step-by-Step: Minimal Python Agent

Step 1	Install: <code>pip install anthropic</code>
Step 2	Define tools as Python functions + JSON schemas
Step 3	Write a system prompt that describes the agent's role and available tools
Step 4	Implement the agent loop: <code>call LLM → check for tool use → execute → loop</code>
Step 5	Handle termination when LLM produces a final text response (no tool call)
Step 6	Add logging, error handling, and a <code>max_iterations</code> safeguard

Minimal Agent Loop (Pseudocode):

```
messages = [{"role": "user", "content": user_goal}]

while True:
    response = llm.call(messages=messages, tools=tool_schemas)
    if response.stop_reason == "end_turn":
        print(response.text); break
    if response.stop_reason == "tool_use":
        for tool_call in response.tool_calls:
            result = execute_tool(tool_call.name, tool_call.args)
            messages.append(tool_result(tool_call.id, result))
```

Recommended Tools for a Starter Agent

- **web_search(query):** Searches the web and returns snippets
- **read_file(path):** Reads a local file and returns its content
- **write_file(path, content):** Writes content to a file
- **run_python(code):** Executes Python in a sandbox and returns output
- **http_request(url, method, body):** Makes arbitrary HTTP API calls

17 Real-world Agent Use Cases

Agentic AI is moving from research into production across enterprise and consumer products. Below are concrete real-world deployments and examples.

Use Case	Description	Example Products
Coding Agent	Write, test, debug, and deploy code autonomously	GitHub Copilot Workspace, Devin, Claude Code
Research Agent	Search web, read papers, synthesize findings	Perplexity Deep Research, OpenAI Deep Research
Data Analysis	Load datasets, write and run code, generate reports	ChatGPT Advanced Data Analysis, Julius AI
Web Browsing	Navigate sites, fill forms, extract information	Operator (OpenAI), Computer Use (Anthropic)
Customer Service	Resolve tickets end-to-end without human handoff	Intercom Fin, Salesforce Agentforce
Sales Automation	Research prospects, draft outreach, log CRM data	Clay, 11x.ai
IT Operations	Monitor alerts, diagnose issues, run remediation	PagerDuty AI, Datadog Bits AI
Legal Automation	Review contracts, identify risks, suggest redlines	Harvey, EvenUp
Financial Analysis	Pull earnings data, run models, generate reports	Viz.ai, BloombergGPT agents

18 Limitations & Safety Guardrails

Autonomous agents operating in the real world carry significant risks. A coding agent that deletes the wrong files or a browsing agent that makes unintended purchases can cause real harm. Safety engineering is not optional.

Key Risks in Agentic Systems

Risk	Description	Example
Irreversible Actions	Agent takes actions that cannot be undone	Deleting files, sending emails, charging cards
Prompt Injection	Malicious content in environment hijacks agent	Web page instructs agent to exfiltrate data
Scope Creep	Agent takes actions beyond its intended scope	Research agent modifies production database
Infinite Loops	Agent loops without making progress	Retries same failing tool call indefinitely
Cascade Failures	Error in one agent corrupts downstream agents	Bad data passed through a multi-agent pipeline
Cost Explosion	Runaway agent makes thousands of API calls	Agent spawns sub-agents that spawn more agents
Data Leakage	Agent sends sensitive data to external services	PII exfiltrated via web search tool

Safety Guardrail Strategies

- **Minimal permissions:** Grant only necessary filesystem/API access (principle of least privilege)
- **Human-in-the-loop:** Require approval for irreversible or high-impact actions
- **Sandboxing:** Run code execution in isolated containers (Docker, E2B, Modal)
- **Action allowlists:** Explicitly enumerate permitted actions; block all others
- **Iteration limits:** Cap max steps/tokens per agent run to prevent runaway loops
- **Input validation:** Sanitize all tool inputs before execution
- **Output validation:** Verify agent outputs meet format and safety criteria before use
- **Monitoring & tracing:** Log every action for audit; alert on anomalies (LangSmith, Arize)
- **Reversibility preference:** Design agents to prefer reversible actions when possible

■■ **Safety Principle:** Treat every agentic system with the security mindset of an external API: assume inputs can be malicious, assume the model can be manipulated, and never grant more access than absolutely necessary for the task.

Summary & Key Takeaways

Part 1: Generative AI	Part 2: Agentic AI
• LLMs generate text by predicting the next token • Transformers + self-attention power modern models • GPT, Claude, Gemini lead the model landscape • Prompt engineering dramatically improves outputs • Gen AI transforms every industry vertical • APIs are the integration layer for applications • Hallucinations require mitigation strategies • Responsible AI is a professional obligation	• Agents autonomously pursue goals over multiple steps • Agents = LLM + Memory + Tools + Planning loop • Function calling enables real-world tool use • LangChain/CrewAI abstract agent complexity • Multi-agent systems tackle complex long-horizon tasks • Memory types: short-term, episodic, semantic, procedural • Agents are in production: coding, research, service • Safety guardrails are non-negotiable for deployment

■ **The Big Picture:** Generative AI gives machines the ability to create; Agentic AI gives them the ability to act. Together, they form the foundation of autonomous AI systems that can reason, plan, use tools, and accomplish complex goals with minimal human supervision.